



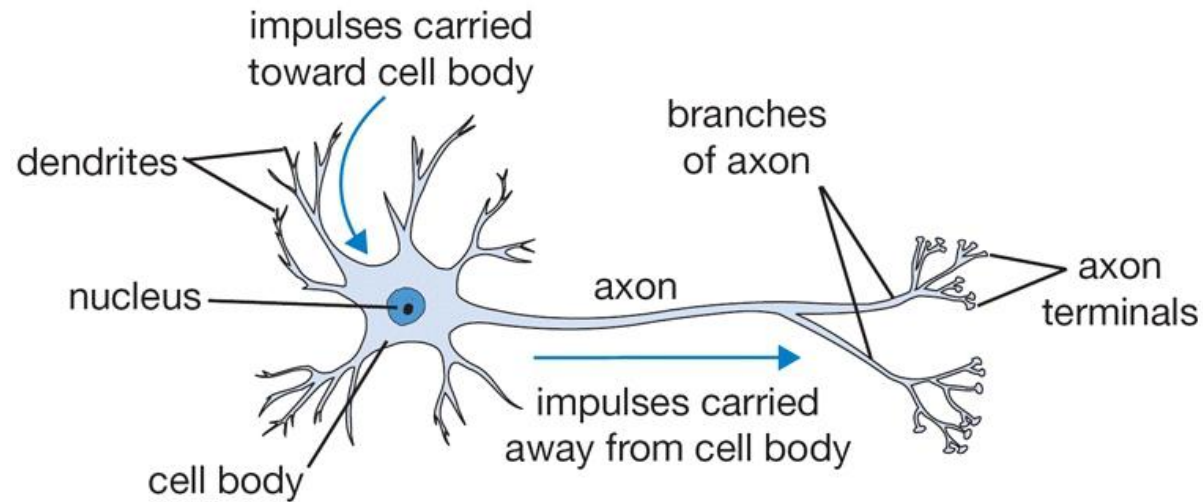
Neural Networks

- Neural Networks
- Backpropagation
- Activation Function
- Hyperparameters
- Learning process

Neural Networks

Biologically inspired by the human brain

- Interconnected cells, signal propagation (synapses)

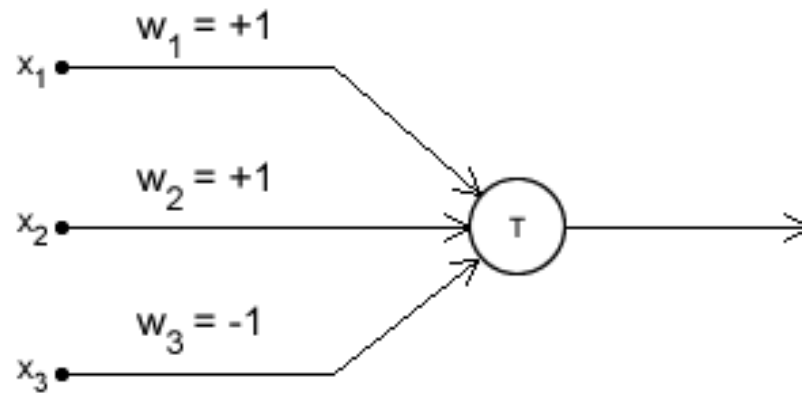


[Image from <http://cs231n.github.io/>]

Neural Networks

The idea of artificial neuron dates back to 1940s

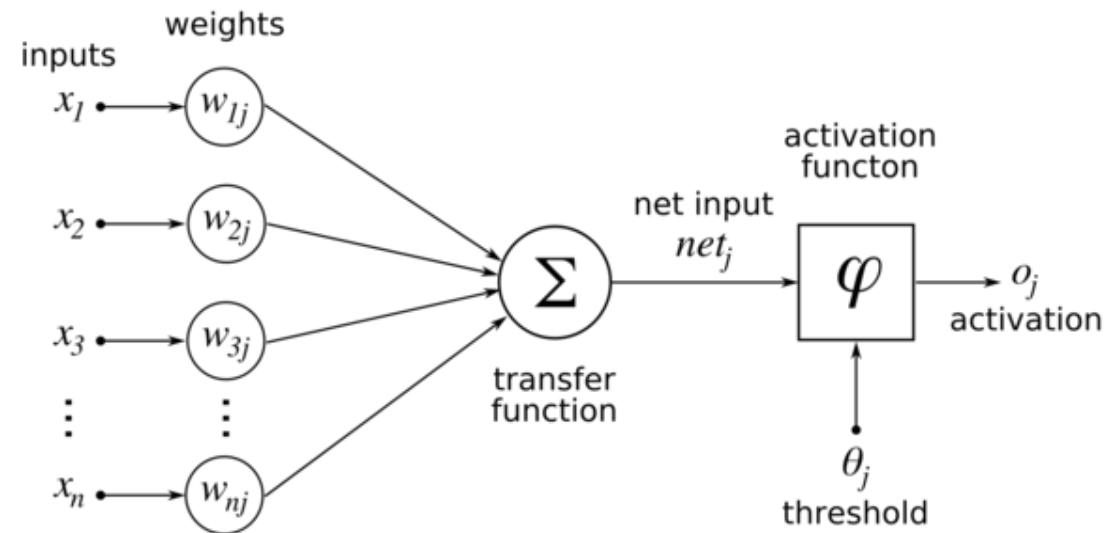
- 1943: McCulloch and Pitts



[Image from <http://aishack.in>]

Neural Networks

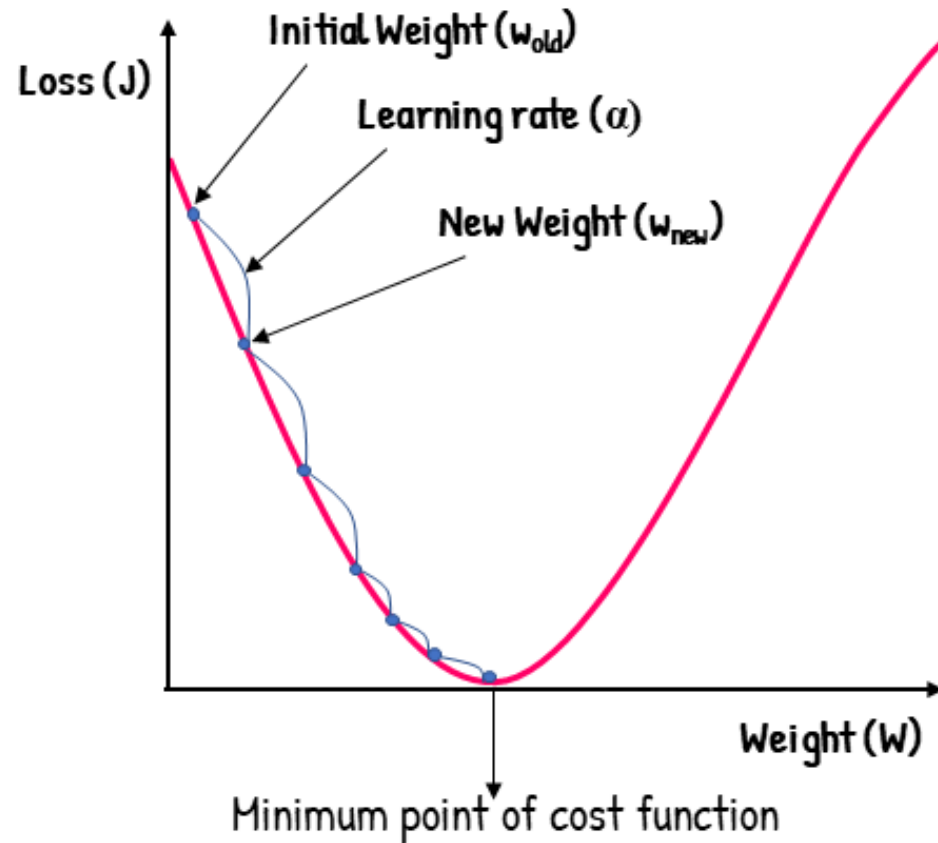
- 1958: Rosenblatt's perceptron



[Image from Wikipedia]

Backpropagation

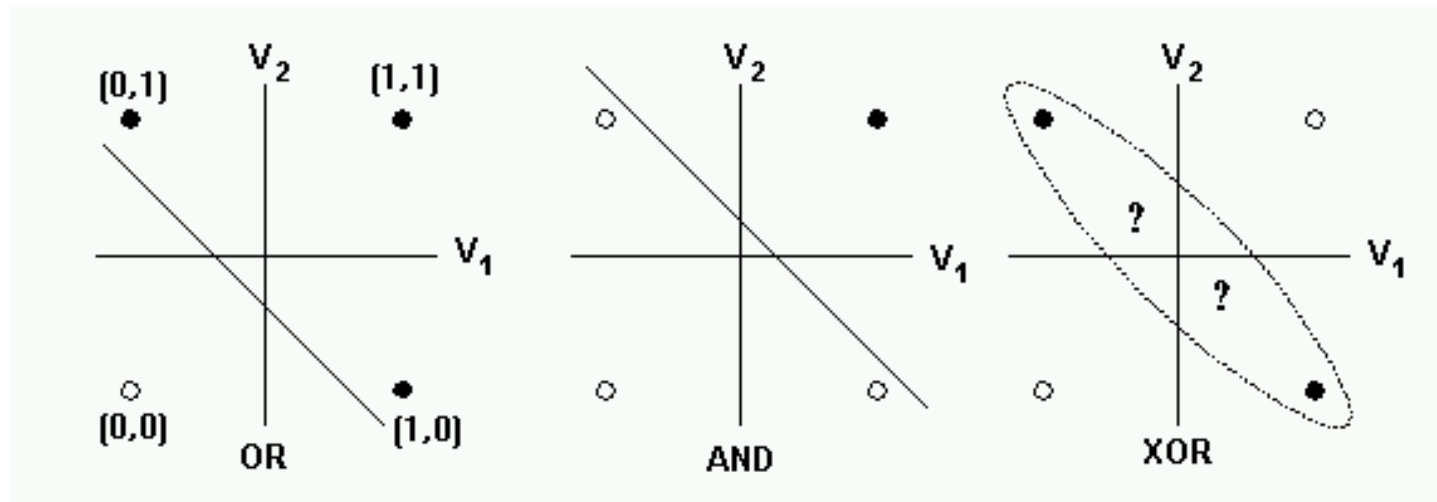
Gradient Descent



$$w_{new} = w_{old} - \alpha \frac{\delta J}{\delta w}$$

Neural Networks

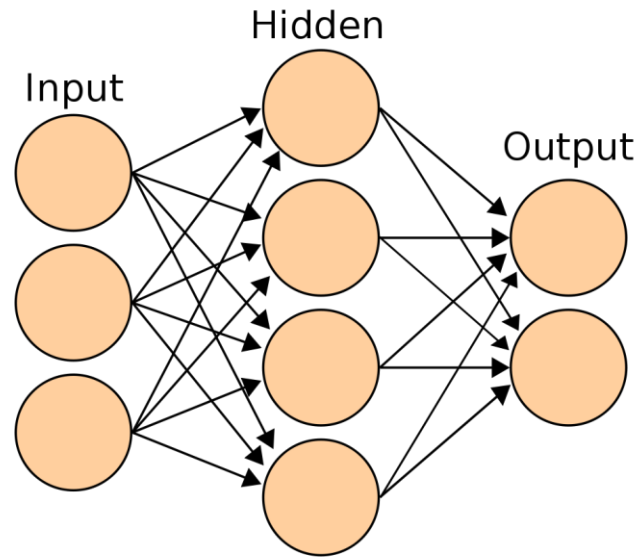
- 1969: Minsky & Papert: limits of the perceptron



[Image from colorado.edu]

Neural Networks

- 1970s-1980s: Multi-Layer Perceptron



[Image from Wikipedia]

Neural Networks



- 1970s-1980s: Multi-Layer Perceptron

The output of a node is the weighted sum of its inputs, to which an **activation function** is applied:

$$o_j^\ell = \sigma \left(\sum_{i=1}^m w_{ij}^\ell o_i^{\ell-1} \right)$$

The activation function must be non linear, otherwise multilayers are useless

Weights are learned so as to minimize the error on the output layer (e.g., the squared error)

This is called the **backpropagation algorithm**

Backpropagation

Backpropagation — Main ideas...

- Initialize the network with **random** weights
- Show the training examples to the network (**epoch**) and **adjust** the weights so that the output matches the desired output (**target**)
- Keep **iterating** over epochs until some **convergence criterion** is met (more on this later)

Backpropagation

Backpropagation — Main ideas...

- The learning phase exploits a **gradient descent** algorithm such as for the perceptron

$$w_{ij}^{t+1} = w_{ij}^t - \eta \frac{\partial E}{\partial w_{ij}}$$

BATCH OR ONLINE AS FOR PERCEPTRON
(More on this later...)

Backpropagation - Loss

Loss functions (some examples for classification)

$y=1, p(y)=1$, correct example, $E = 0$
 $y=1, p(y)=0$, wrong example, $E = +\ln f$

- Binary cross-entropy

$$E = -\frac{1}{N} \sum_{i=1}^N y_i \log(p(y_i)) + (1 - y_i) \log(1 - p(y_i))$$

- Multinomial cross-entropy / categorical cross-entropy

$$E = -\frac{1}{N} \sum_{i=1}^N \sum_{j=1}^K y_{ij} \log(p(y_{ij}))$$

K = num. classes



Backpropagation - Output

Output nodes

- Binary classification (1 output node)

$$o^L = \sigma(z) = \frac{1}{1 + e^{-z}}$$

SIGMOID
(probability of class 1)

- Multinomial classification (k output nodes)

$$o_i^L = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

SOFTMAX
(probabilities over classes)

Backpropagation - Loss

- Loss functions (some examples for regression)

Residual Sum of Squares (RSS)

$$RSS = \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

where $\hat{y}_i$ is computed as

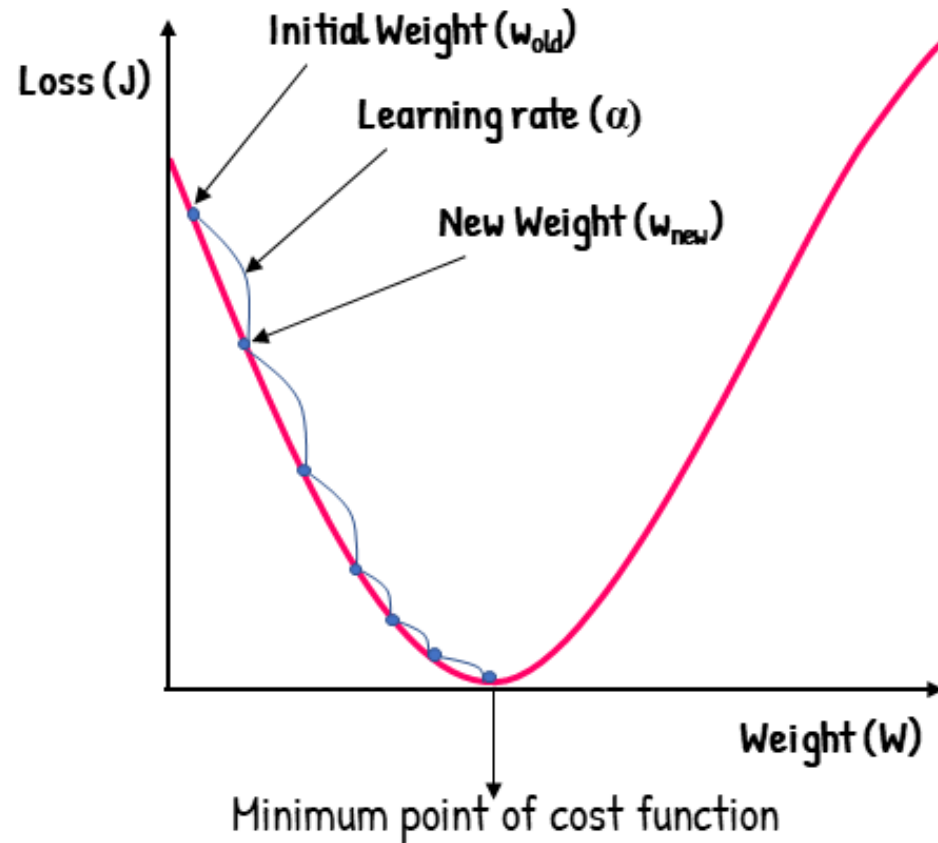
$$o_j^\ell = \sigma \left(\sum_{i=1}^m w_{ij}^\ell o_i^{\ell-1} \right)$$

$\hat{y}_i$ = last output

Last output can be sigmoid?

Backpropagation

Gradient Descent



$$w_{new} = w_{old} - \alpha \frac{\delta J}{\delta w}$$

Backpropagation

Backpropagation — High level code...

1. **init** network weights
2. **repeat**
3. **for each** training pair (x, y)
 - 3a. prediction = **compute output** (w, x)
 - 3b. **compute error** $(\text{prediction} - y)$ at the output units
 - 3c. **propagate** error from output layer to hidden layer
 - 3d. **propagate** error from hidden layer to input layer
 - 3e. update all weights by **gradient step**
4. **until** stopping criterion (more on this later)

Backpropagation

Batch vs. **stochastic** gradient descent

- In general, the gradient is obtained as the **sum** of gradients computed for all the **training examples**
- In principle, we could also update the vector of parameters **after each example** rather than after all the training set
- Update after each example is called **stochastic** update
- **Faster** training, which can help **avoiding** local minima
 - Helps avoiding positive and negative example cancelling each other
- **Mini-batch**

Activation functions

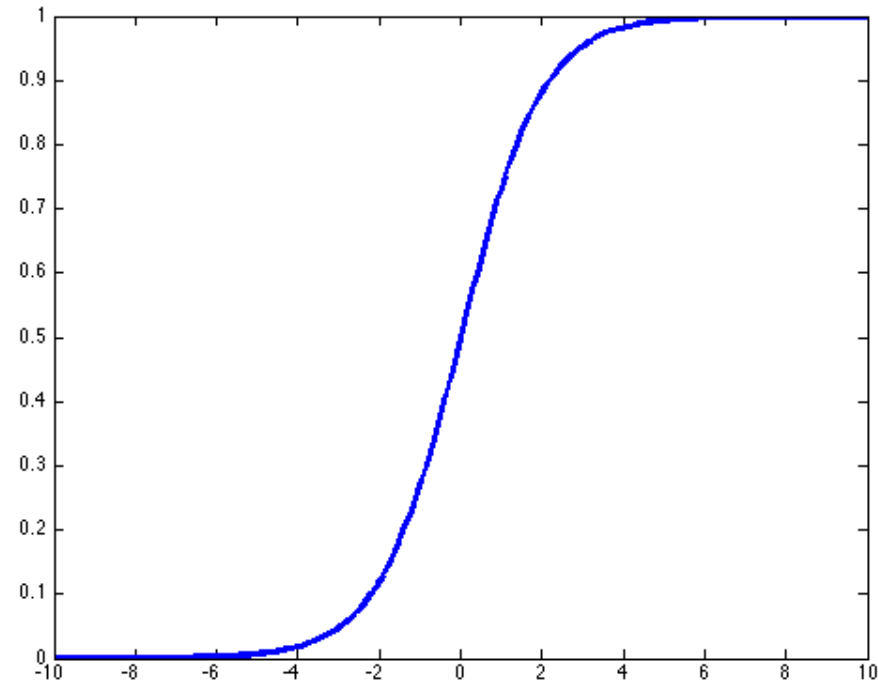
Examples of activation functions:

- Sigmoid
 - Hyperbolic Tangent (Tanh)
 - Rectifier (ReLU — Rectified Linear Unit)
 - ...
-
- CLASSIC MODELS
- DEEP MODELS

Activation functions

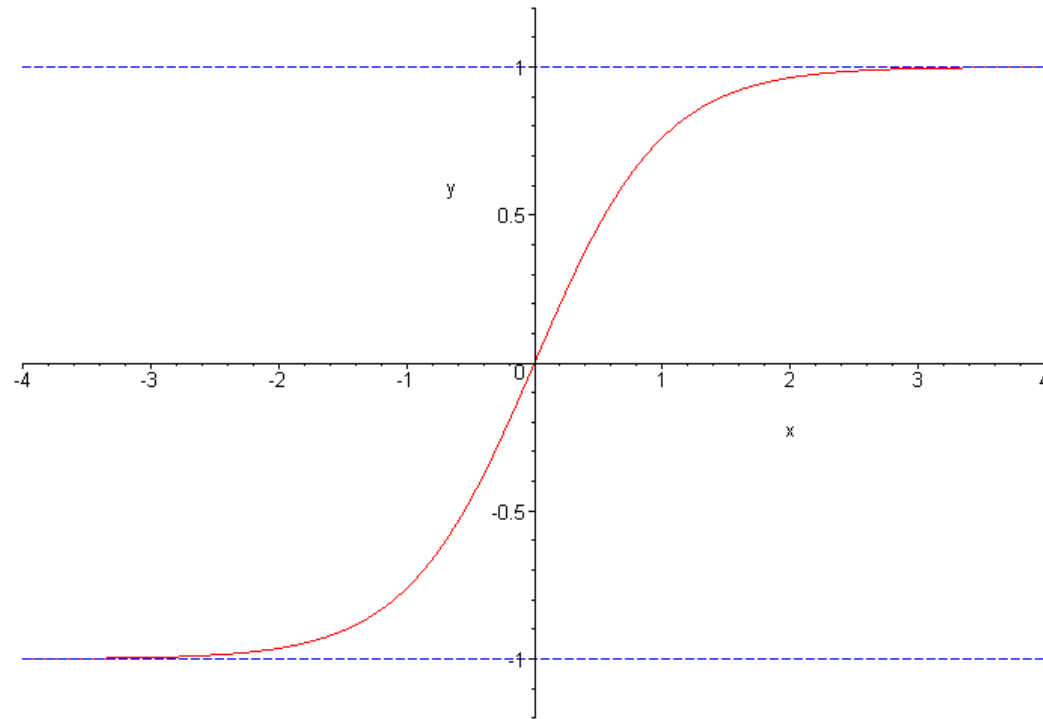
Sigmoid

$$\sigma(t) = \frac{1}{1+e^{-t}}$$



Neural Activation functions

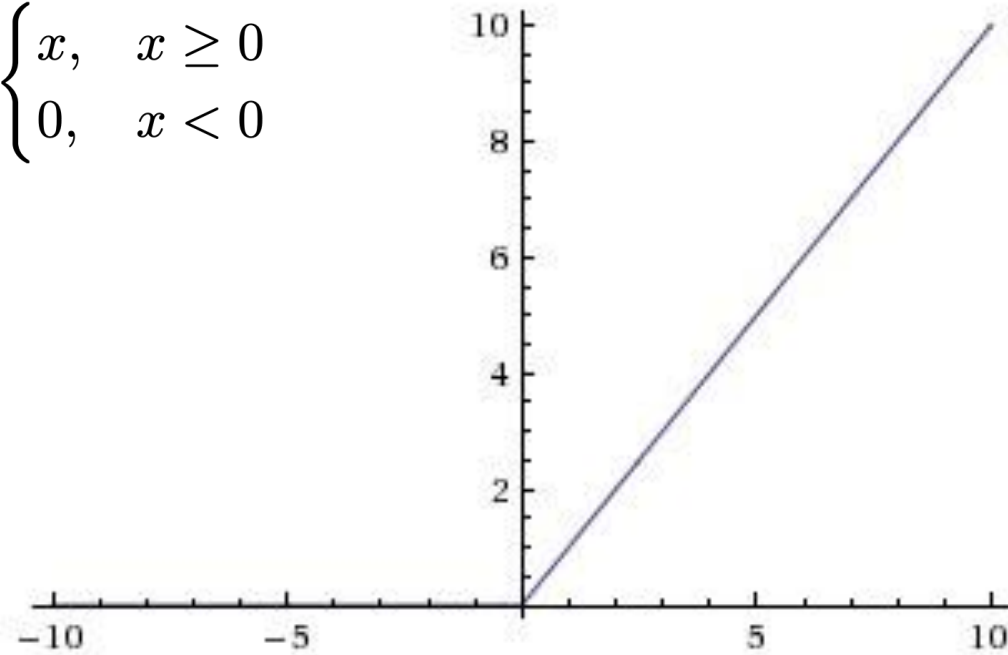
Hyperbolic Tangent



Activation functions

Rectifier

$$\text{ReLU}(x) = \begin{cases} x, & x \geq 0 \\ 0, & x < 0 \end{cases}$$



Non-linearity



These activation functions are all **non-linear**

Neural Networks can learn **non-linear dependencies** between input and output, **without any assumption** on the shape of the function

Often, networks with more layers (**deep learning**) are more powerful models (but learning is more costly)

Vanishing gradients



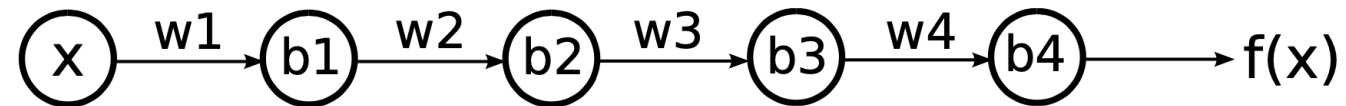
Networks with many (more than 1-2) hidden **layers** and **sigmoid** (a little less with **tanh...**) as activation function suffer of the problem of **vanishing gradients**

Gradients tend to **vanish** (or explode) as we backpropagate errors from the output to the input...

This is mainly due to **composition of derivatives...**

Vanishing gradients

Consider a neural network with 4 layers and a **single unit** in each layer.



We name x the input, w_j and b_j the weight and bias of layer j .
The output of each layer can be computed as:

$$z_j = \sigma(w_j z_{j-1} + b_j)$$

where $z_0 = x$.

[Example by M. Nielsen]

Vanishing gradients

chain rule

$$\frac{dy}{dx} = \frac{dy}{du} \frac{du}{dx}$$

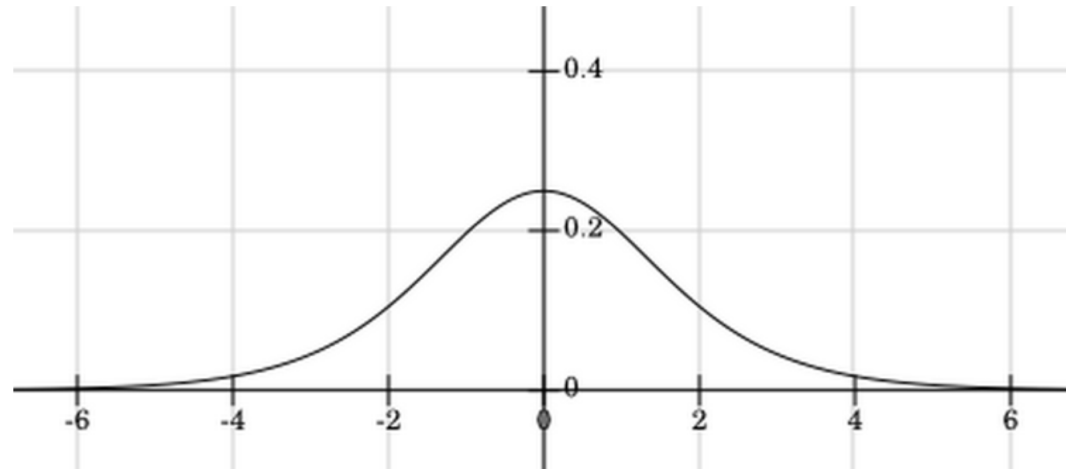
Let us compute the gradient of f with respect to b_1 and b_3 :

$$\frac{\partial f}{\partial b_1} = \sigma'(z_1) \cdot w_2 \cdot \sigma'(z_2) \cdot w_3 \cdot \sigma'(z_3) \cdot w_4 \cdot \sigma'(z_4) \cdot \frac{\partial f}{\partial z_4}$$

$$\frac{\partial f}{\partial b_3} = \sigma'(z_3) \cdot w_4 \cdot \sigma'(z_4) \cdot \frac{\partial f}{\partial z_4}$$

Vanishing gradients

If we now consider the derivative of the sigmoid function:



we have that, with typical weight initialization (randomly sampled from a Gaussian with mean 0 and standard deviation 1):

$$|w_j \sigma'(z_j)| < \frac{1}{4}$$

Vanishing gradients

Now going back to the gradient computation:

$$\frac{\partial f}{\partial b_1} = \sigma'(z_1) \cdot w_2 \cdot \sigma'(z_2) \cdot w_3 \cdot \sigma'(z_3) \cdot w_4 \cdot \sigma'(z_4) \cdot \frac{\partial f}{\partial z_4}$$

$$\frac{\partial f}{\partial b_3} = \sigma'(z_3) \cdot w_4 \cdot \sigma'(z_4) \cdot \frac{\partial f}{\partial z_4}$$

Lower layers will tend to have either vanishing or exploding gradients !

Exhaustive analysis for Recurrent Neural Networks in [Hochreiter et al., 2001]

Hyperparameters

When creating a neural network, one has to choose:

- the number of **layers**
- the number of **hidden units** in each layer
- the **number of epochs**
- the **learning rate**...?

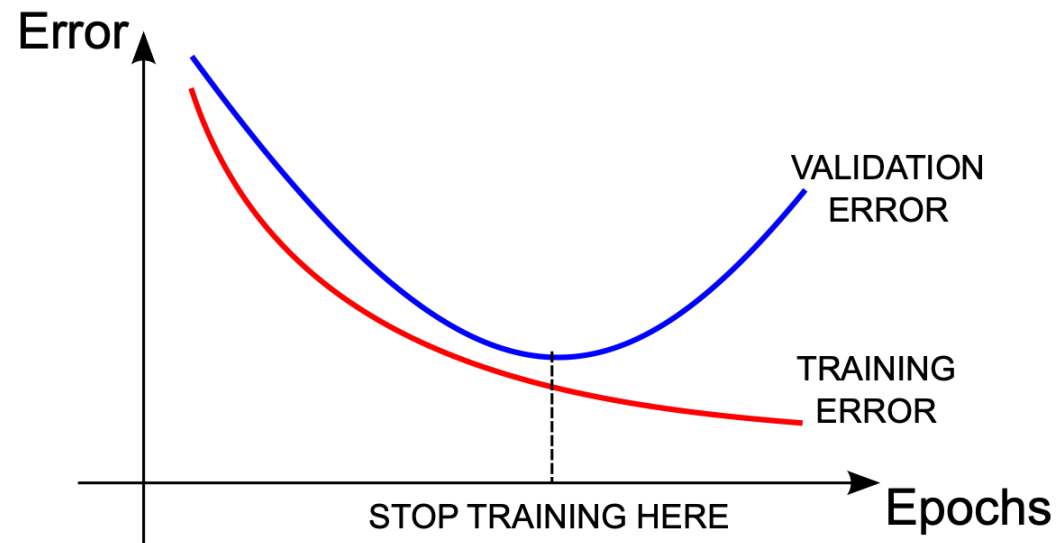
Hyperparameters optimization via validation set!

Hyperparameters

- The number of **hidden layers** has long been 1-2, but now much deeper networks are employed (but in that case **use ReLU activation**)
- The number of **hidden units** in each layer is a set of hyper-parameters that should be chosen according to the **validation** error

Overfitting

Given a network with fixed architecture, the **number of epochs** is chosen following the **early stopping** principle, using a **validation set**



Overfitting

One can also wait for a few epochs to see whether the validation error starts decreasing again (**patience**)

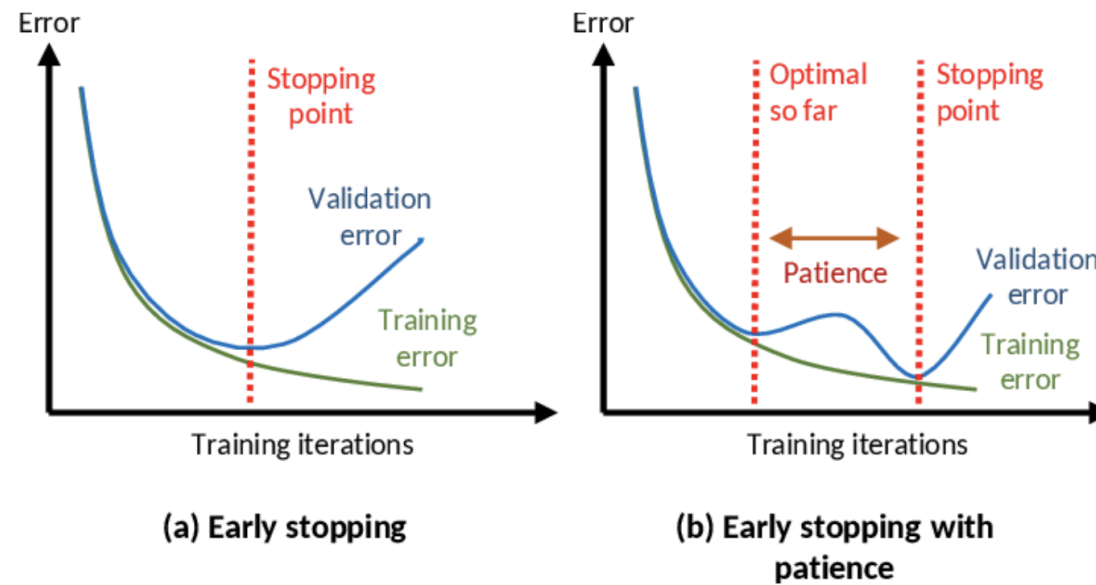


Figure by [Lore et al., 2016]

Learning process



Choosing the learning rate is **tricky**

- Too small: slow convergence
- Too large: divergence (numerically unstable)

This simple method is **obsolete**, we have nowadays more sophisticated optimizers that drive learning
The key idea is to **adjust** the learning rate across the epochs, and have a different one for **each** parameter

Learning process



Stochastic Gradient Descent

$$w_{ij}^{t+1} = w_{ij}^t - \eta \frac{\partial E}{\partial w_{ij}}$$

Momentum

$$w_{ij}^{t+1} = w_{ij}^t - \eta \frac{\partial E}{\partial w_{ij}} - \mu \left(\frac{\partial E}{\partial w_{ij}} \right)^{t-1}$$

Learning process

Adam

Key idea: one learning rate for each parameter!

The diagram illustrates the Adam optimizer's update rules. It features three equations with blue arrows pointing to specific terms: 'INITIAL LEARNING RATE' points to η in the Δw_t equation; 'GRADIENT' points to g_t in the same equation; and 'EXPONENTIAL AVERAGE OF SQUARES OF GRADIENTS' points to s_t in the s_t update equation. A blue box on the right states 'THIS IS THE MOST WIDELY EMPLOYED OPTIMIZER NOWADAYS'.

$$w_{t+1} = w_t + \Delta w_t$$
$$\Delta w_t = -\eta \frac{\nu_t}{\sqrt{s_t + \epsilon}} g_t$$
$$\nu_t = \beta_1 \nu_{t-1} - (1 - \beta_1) g_t$$
$$s_t = \beta_2 s_{t-1} - (1 - \beta_2) g_t^2$$

INITIAL LEARNING RATE

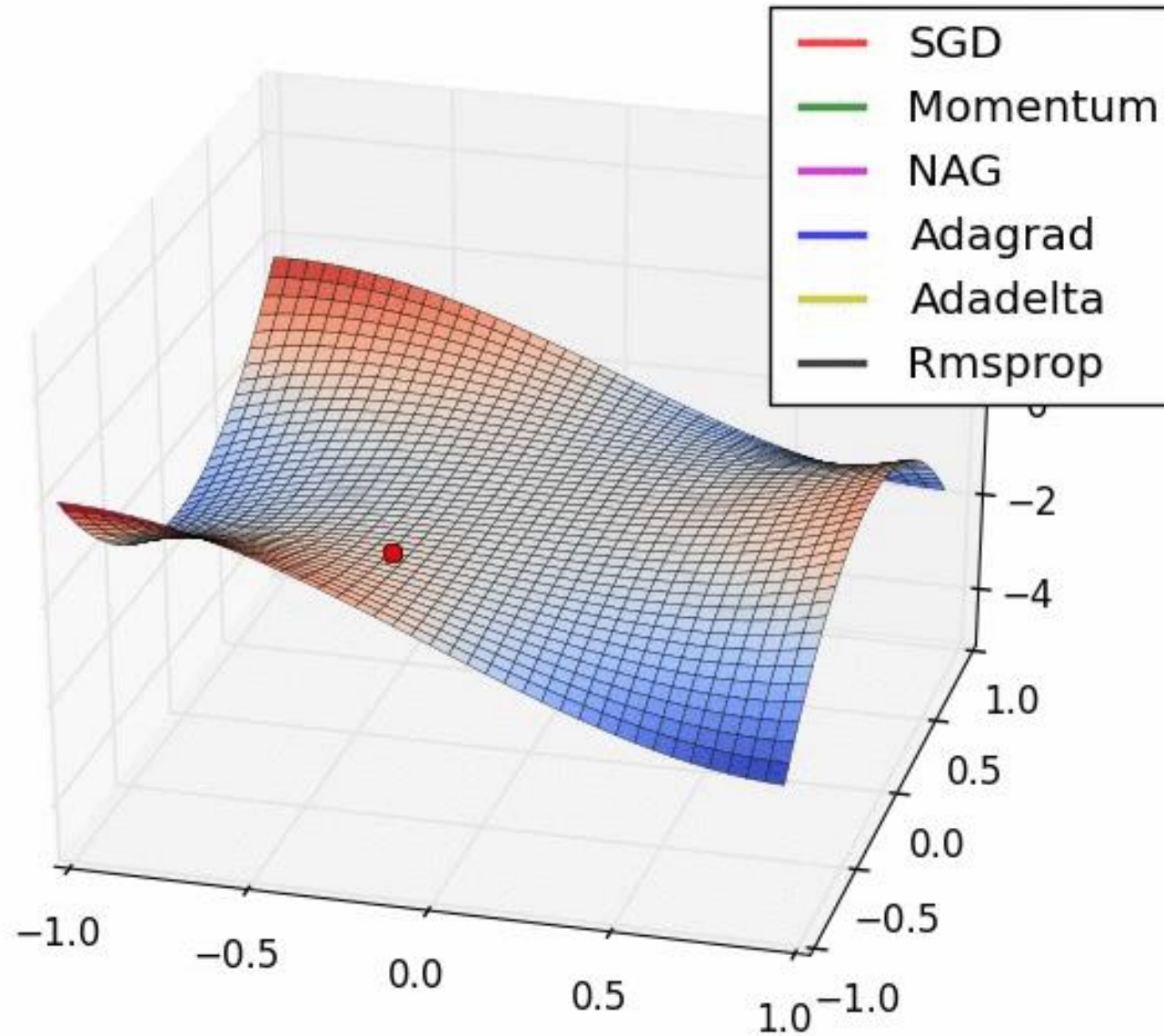
GRADIENT

EXPONENTIAL AVERAGE OF SQUARES OF GRADIENTS

THIS IS THE MOST WIDELY EMPLOYED OPTIMIZER NOWADAYS

<https://towardsdatascience.com/a-visual-explanation-of-gradient-descent-methods-momentum-adagrad-rmsprop-adam-f898b102325c>

SGd vs Momentum vs. Rmsprop



Neural Networks



Some tricks of the trade

- One-hot encoding
- Input range and standardization (zero-mean input)
- Smart weight initialization

Many packages for neural network with Python

- Scikit-learn (limited)
- Keras
- Tensorflow
- PyTorch

Deep Learning

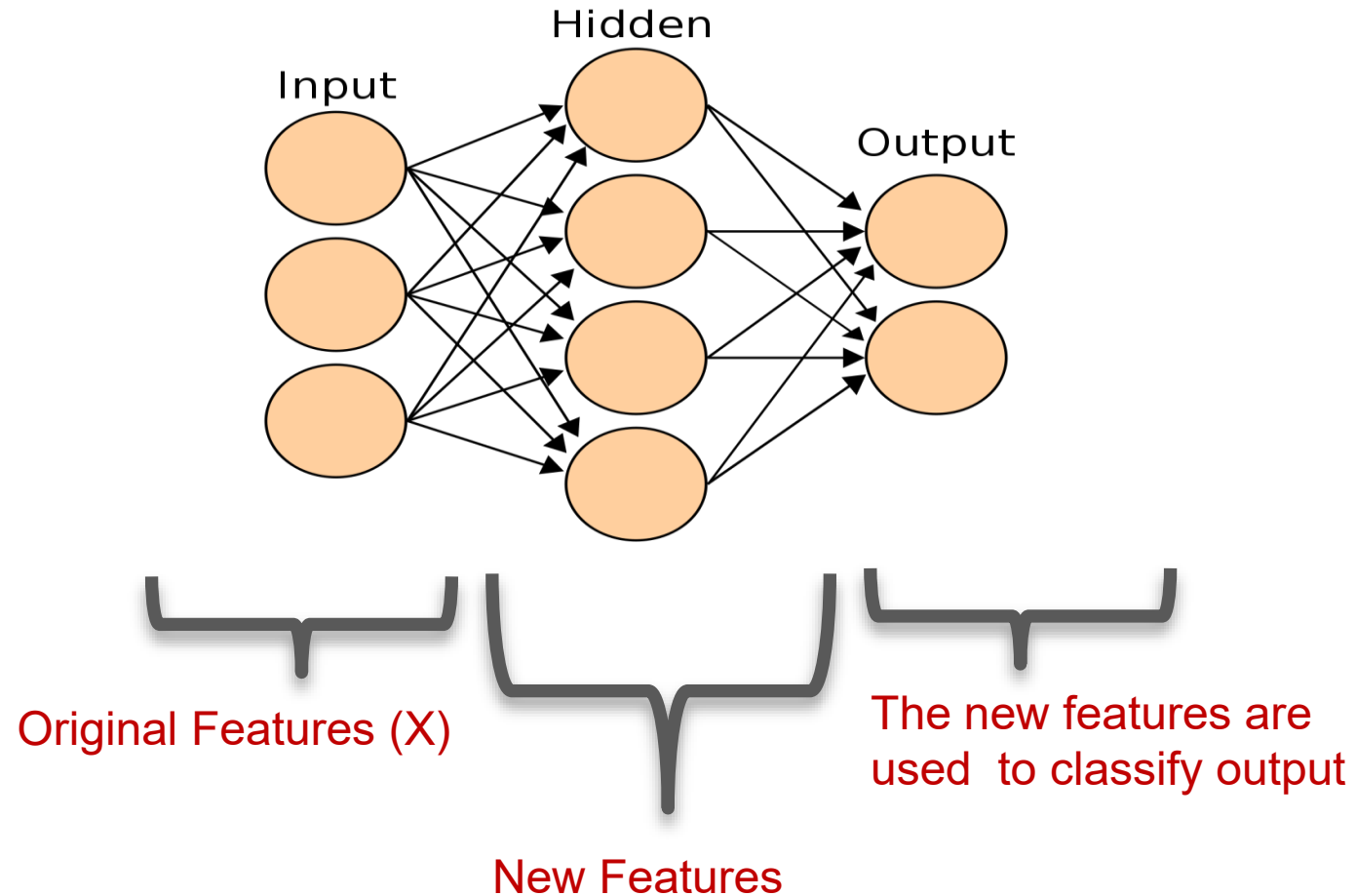


Deep learning is a **hot topic** in artificial intelligence and probably in computer science in general

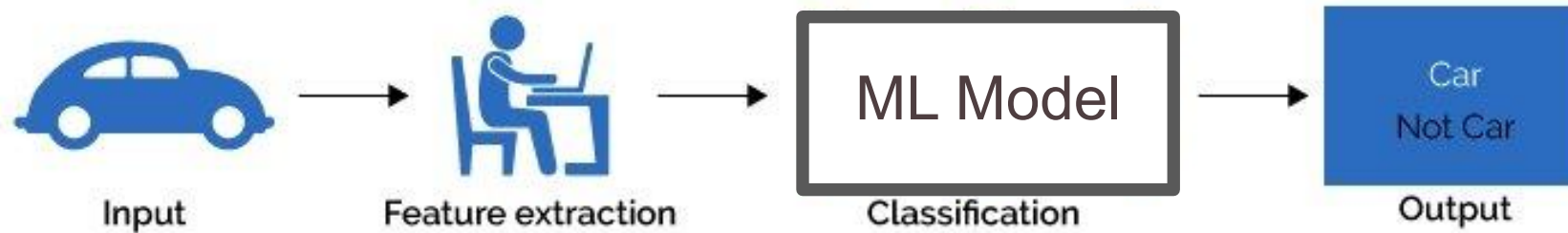
It does **not** mean just “neural networks with many layers” but it includes a wide collection of **architectures (i.e. neurons’ configuration)** useful for many purposes.

- Convolutional neural networks
- Recurrent neural networks
- Recursive neural networks
- Attention-based networks
- Transformers
- ...

Feature Creation



Machine Learning



Neural network / deep learning

